

Tyler Harris
599 Edge AI and Robotics
Module 4 Jetbot ROS and Gazebo

Overview:

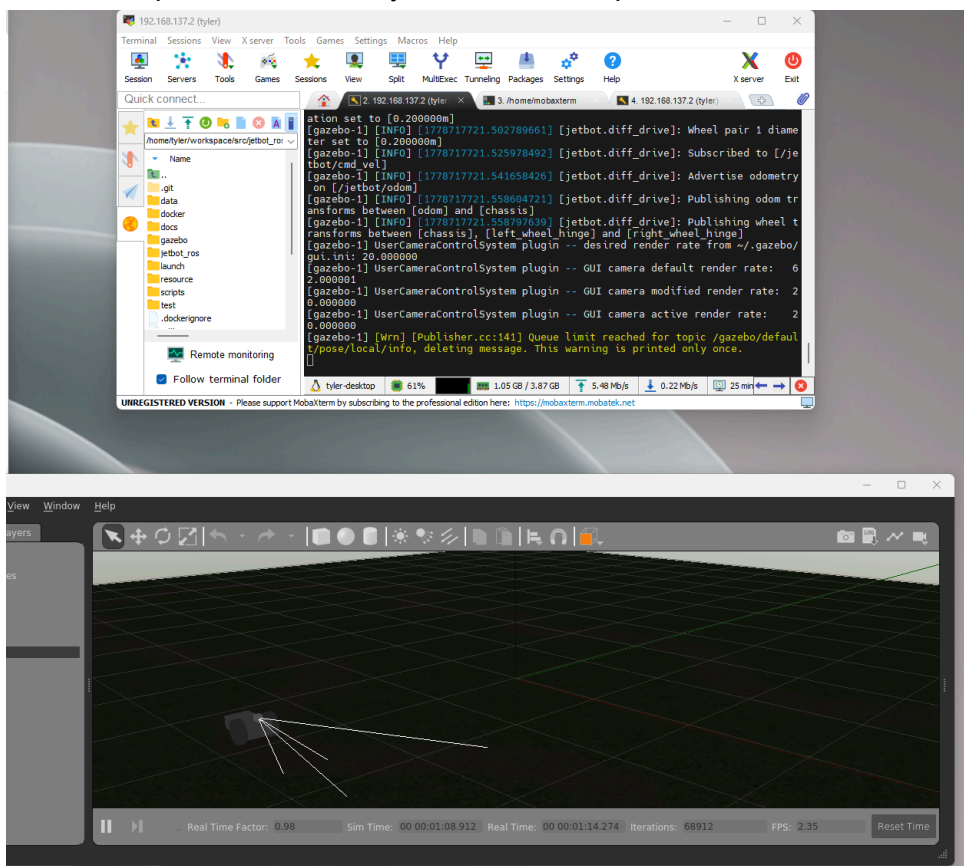
This lab involves running a docker container with a JetBot simulation built with Gazebo. The purpose is to train the JetBot on a pathfinding task using data collected inside the simulation. Using a simulated camera samples are collected and a provided training script is used to train a model to stay on the path.

Setup:

After installing the needed container and cloning the provided repository, the simulation is run on the Jetson Nano and streamed using MobaXterm to the host machine.

Launching the Simulation:

Using the gazebo_world [launch.py](#) script provided from the repo, the simulation begins. With just the startup script the environment cannot be interacted with, there is a simple path visible on a flat 2d plane and the body of the JetBot is present.



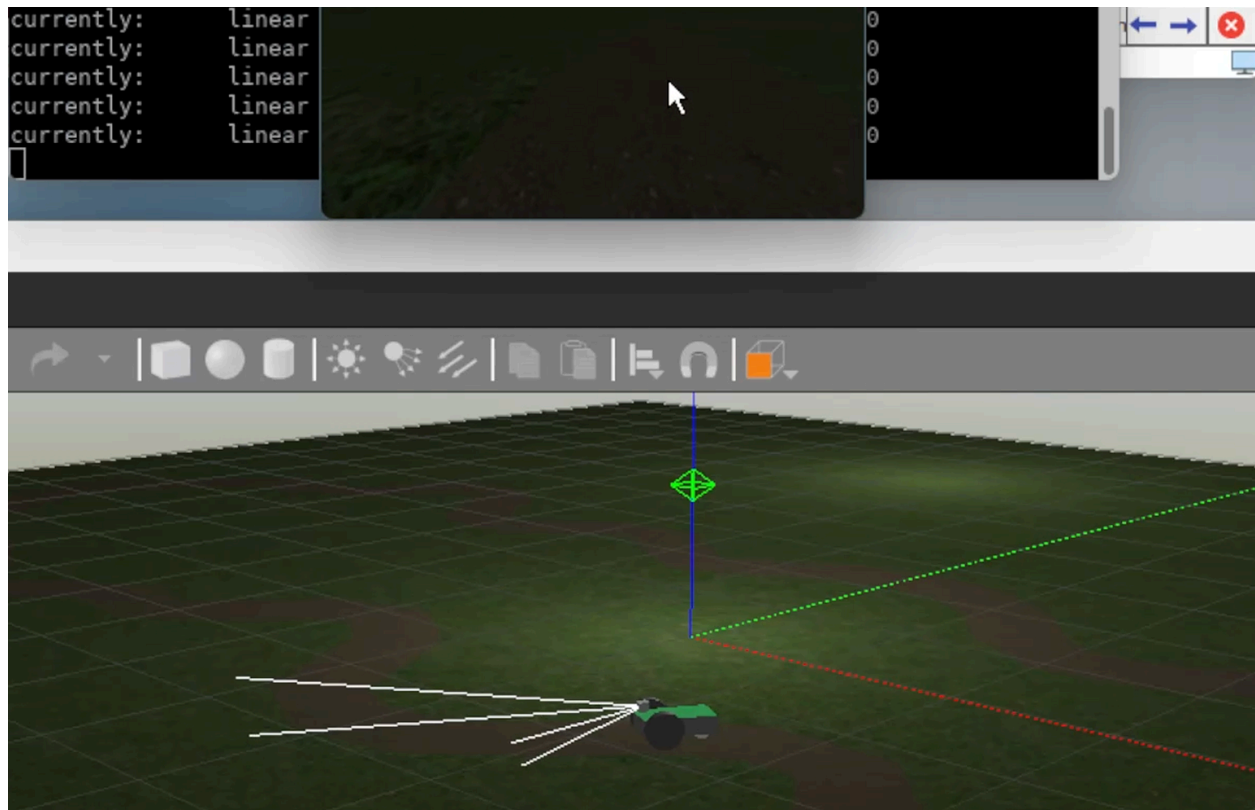
Controlling the JetBot:

Using another ssh terminal, the teleop_keyboard [launch.py](#) script can be run to allow interaction with the JetBot via keyboard controls (WASD). A video of controlling the JetBot can be found

under /media. The controls are fairly intuitive, the movement is inertial so a key press in any direction persists at a continuous rate until a counter-movement is given, e.g. pressing D spins the JetBot clockwise until A is pressed to counter the movement.

Collecting Data:

Using the provided data_collection.[launch.py](#) script, it creates an environment similar to the first, however the key 'C' can be used to capture images. This prompts a popup to appear in which you manually select the center point of the path seen by the simulated camera in order to train the JetBot. There are minimal instructions in the lab as to how many samples to collect, however they do advise to capture some images where the JetBot is off-track to improve performance.



Training the Model:

After collecting the data (44 images in this lab), the provided [train.py](#) file is used on the created dataset. After completion a best_model.pth file is created that can be run to test the performance.

Trained Model Performance:

There is a video of the trained model located under /media. In this case the trained model performed fairly poorly, which is not entirely unexpected given 44 total training images. It initially begins by staying on the track however as soon as it gets off track it begins to spin somewhat aimlessly. During this spin it crosses back over the track and is then able to temporarily maintain

course again, but only for a short time. This is very likely due to the limited data available, and even fewer instances of being off course for it to accurately learn to maintain the course. I could have provided many more samples however the training process in this simulation is somewhat arduous. The framerate of the simulation being run on a Nano is less than ideal, and there is a noticeable amount of latency between the host and Nano. The increased latency in combination with the inertial movement meant that in order to accurately stay on track, I had to move the nano at the slowest pace which was very tedious for navigating the curved track. That being said even with such few instances the model could make some kind of attempt to stay on the course, which is impressive.

